

ServletContext

Simple idea: `ServletContext` is the common/shared information and runtime object for the **whole web application**.

1. Definition

`ServletContext` is an object provided by the **Servlet container** that represents the **entire web application**.

It is used to:

- Share configuration across servlets
- Share data between servlets
- Access application resources
- Get application information
- Obtain a `RequestDispatcher`

Simple Definition

```
ServletContext = Common / Shared information for the WHOLE application
```

2. ServletConfig vs ServletContext — Simple Understanding

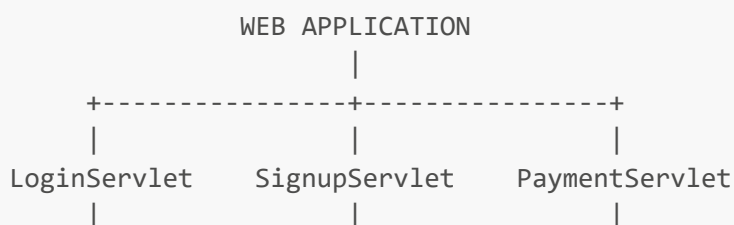
The easiest way to understand the difference is:

```
ServletConfig
  ↓
Information for ONE servlet

ServletContext
  ↓
Information for the WHOLE application
```

Example

Imagine a web application containing:





If `LoginServlet` stores something in `ServletContext`, other servlets such as `SignupServlet` and `PaymentServlet` can also access it.

3. Why Do We Need ServletContext?

Suppose an application has 10 servlets.

All 10 servlets may need common information such as:

- Application name
- Company name
- Admin email
- Website name
- Database-related configuration
- Shared application data

Instead of storing the same information separately in every servlet, we can keep common information at the **application level** using `ServletContext`.

Without ServletContext

```
// LoginServlet
String company = "ABC";

// SignupServlet
String company = "ABC";

// PaymentServlet
String company = "ABC";

// ReportServlet
String company = "ABC";
```

Problem

The same information is repeated in multiple places.

With ServletContext

```
ServletContext
├── company = "ABC"
```

```
LoginServlet    → gets company
SignupServlet   → gets company
PaymentServlet  → gets company
ReportServlet   → gets company
```

There is now **one common place** for application-wide information.

4. Real-Life Example

Think about a **college**.

A college may have multiple departments:

- Computer
- Mechanical
- Civil
- Electrical

The college has common information such as:

- College name
- College address
- Principal name

This information belongs to the **whole college**, not to one particular department.

Similarly:

```
ServletConfig
  → Information for ONE servlet

ServletContext
  → Information for the WHOLE web application
```

Easy Analogy

```
College
|
+-- Computer Department
+-- Mechanical Department
+-- Civil Department
+-- Electrical Department

Common college information
↓
ServletContext
```

5. ServletConfig vs ServletContext

Point	ServletConfig	ServletContext
Scope	One servlet	Whole application
Purpose	Servlet-specific configuration	Application-wide configuration/data
Access	<code>getServletConfig()</code>	<code>getServletContext()</code>
Parameters	<code>init-param</code>	<code>context-param</code>
Sharing	Generally for that servlet	Can be accessed by multiple servlets
Example	Login servlet's email	Application name/common email
Objects	One config object for each servlet	One context for the web application

Easy Memory Trick

```
ServletConfig
  ↓
ONE servlet

ServletContext
  ↓
WHOLE application
```

6. How to Get ServletContext

There are two common ways to obtain `ServletContext`.

Method 1 — Directly

```
ServletContext context = getServletContext();
```

Method 2 — Through ServletConfig

```
ServletConfig config = getServletConfig();

ServletContext context = config.getServletContext();
```

Most Common

Usually, you will see:

```
ServletContext context = getServletContext();
```

7. Important ServletContext Methods

Method	Purpose
<code>getInitParameter(String name)</code>	Gets an application-wide initialization parameter
<code>getInitParameterNames()</code>	Gets all application-wide initialization parameter names
<code>getAttribute(String name)</code>	Gets a shared application attribute
<code>setAttribute(String name, Object value)</code>	Stores a shared application attribute
<code>removeAttribute(String name)</code>	Removes a shared application attribute
<code>getServletContextName()</code>	Gets the name of the web application
<code>getRequestDispatcher(String path)</code>	Gets a <code>RequestDispatcher</code> using the <code>ServletContext</code>
<code>getResourceAsStream(String path)</code>	Reads a resource from the web application
<code>log(String message)</code>	Writes a message to the server log

Most Important Methods to Remember

```
getInitParameter()  
getAttribute()  
setAttribute()  
removeAttribute()  
getRequestDispatcher()
```

8. context-param

`ServletContext` can have **application-wide initialization parameters**.

These are commonly defined in `web.xml`.

Example

```
<web-app>

  <context-param>
    <param-name>companyName</param-name>
    <param-value>ABC Company</param-value>
  </context-param>

</web-app>
```

This parameter belongs to the **whole application**.

Reading the Parameter in a Servlet

```
ServletContext context = getServletContext();

String company = context.getInitParameter("companyName");

System.out.println(company);
```

Output

ABC Company

Flow

```
web.xml
  |
  ↓
context-param
  |
  ↓
ServletContext
  |
  ↓
Multiple Servlets
```

9. context-param vs init-param

This is one of the most important differences to understand.

init-param

Used for configuration belonging to **ONE servlet**.

Example

```
<servlet>

    <servlet-name>LoginServlet</servlet-name>

    <servlet-class>LoginServlet</servlet-class>

    <init-param>
        <param-name>email</param-name>
        <param-value>login@gmail.com</param-value>
    </init-param>

</servlet>
```

The above parameter belongs to `LoginServlet`.

context-param

Used for configuration belonging to the **WHOLE application**.

Example

```
<context-param>
    <param-name>company</param-name>
    <param-value>ABC Company</param-value>
</context-param>
```

This parameter can be accessed by multiple servlets.

Easy Memory

```
init-param
  ↓
ONE servlet

context-param
  ↓
WHOLE application
```

10. Sharing Data Using ServletContext

`ServletContext` can also be used to share data between servlets.

Servlet 1 — Store Data

```
ServletContext context = getServletContext();  
  
context.setAttribute("username", "Harsh");
```

The value "Harsh" is stored in the application-wide `ServletContext`.

Servlet 2 — Get Data

```
ServletContext context = getServletContext();  
  
String username = (String) context.getAttribute("username");  
  
System.out.println(username);
```

Output

```
Harsh
```

Flow

```
Servlet 1  
  |  
  ↓  
setAttribute()  
  |  
  ↓  
ServletContext  
  |  
  ↓  
getAttribute()  
  |  
  ↓  
Servlet 2
```

Important Methods

```
setAttribute()  
  → Store data
```

```
getAttribute()
  → Get data

removeAttribute()
  → Remove data
```

11. ServletContext vs `.env`

They may look similar because both can be involved with configuration, but they are **not the same thing**.

`.env`

An `.env` file is commonly used for application/environment configuration such as:

```
DB_HOST
DB_USER
DB_PASSWORD
API_KEY
PORT
```

ServletContext

`ServletContext` is used inside the servlet application for:

- Application-wide configuration
- Shared application attributes
- Accessing application resources
- `RequestDispatcher`
- Servlet/application information

Simple Difference

```
.env
  ↓
Configuration outside the application/runtime environment

ServletContext
  ↓
Application-level object provided by the Servlet container
```

12. Why Can't We Completely Replace ServletContext with `.env`?

Because `ServletContext` does much more than store configuration values.

`.env` Can Store

```
DB_HOST=localhost
DB_NAME=mydb
COMPANY=ABC
```

But `.env` does not provide runtime methods such as:

```
setAttribute()
getAttribute()
removeAttribute()
getRequestDispatcher()
getResourceAsStream()
```

`ServletContext` is an object available at runtime inside the Servlet application.

It can:

1. Store application-wide configuration
2. Store shared runtime data
3. Share data between servlets
4. Access application resources
5. Provide `RequestDispatcher`
6. Provide application information

Therefore

```
.env
=
Mainly configuration

ServletContext
=
Configuration
+
Application-wide runtime features
```

13. Important: `.env` Is Better for Secrets

Sensitive values such as:

- Database passwords

- API secrets
- Secret keys

should not normally be placed as plain text inside `web.xml`.

For modern applications, **environment variables or secure secret-management systems** are generally preferred for sensitive configuration.

Example `.env`

```
DB_USER=root
DB_PASSWORD=secret
```

ServletContext Is Better Understood For

```
Application-wide settings
Shared runtime objects/data
Servlet application features
```

Important: The exact way environment variables are loaded into a Java Servlet application depends on the server, deployment setup, and application configuration. `.env` and `ServletContext` are different mechanisms.

14. Final Summary

Core Concepts

```
ServletConfig
  → Configuration for ONE servlet

ServletContext
  → Configuration/data/features for the WHOLE application

init-param
  → One servlet

context-param
  → Whole application

setAttribute()
  → Store shared runtime data

getAttribute()
  → Get shared runtime data
```

```
removeAttribute()  
    → Remove shared runtime data  
  
.env  
    → Mainly environment/application configuration  
  
ServletContext  
    → Application-wide runtime object  
    + configuration  
    + features
```

15. One-Minute Revision

What is `ServletContext`?

`ServletContext` represents the **whole web application**.

What is `ServletConfig`?

`ServletConfig` represents configuration for **one servlet**.

What is `init-param`?

Configuration for **one servlet**.

What is `context-param`?

Configuration for the **whole application**.

How do you get `ServletContext`?

```
ServletContext context = getServletContext();
```

How do you store shared data?

```
context.setAttribute("key", value);
```

How do you retrieve shared data?

```
context.getAttribute("key");
```

How do you remove shared data?

```
context.removeAttribute("key");
```

Easy Memory Trick

ServletConfig

ONE servlet

ServletContext

WHOLE application

.env

Environment config

Remember

Config = ONE servlet

Context = WHOLE application

.env = Environment configuration